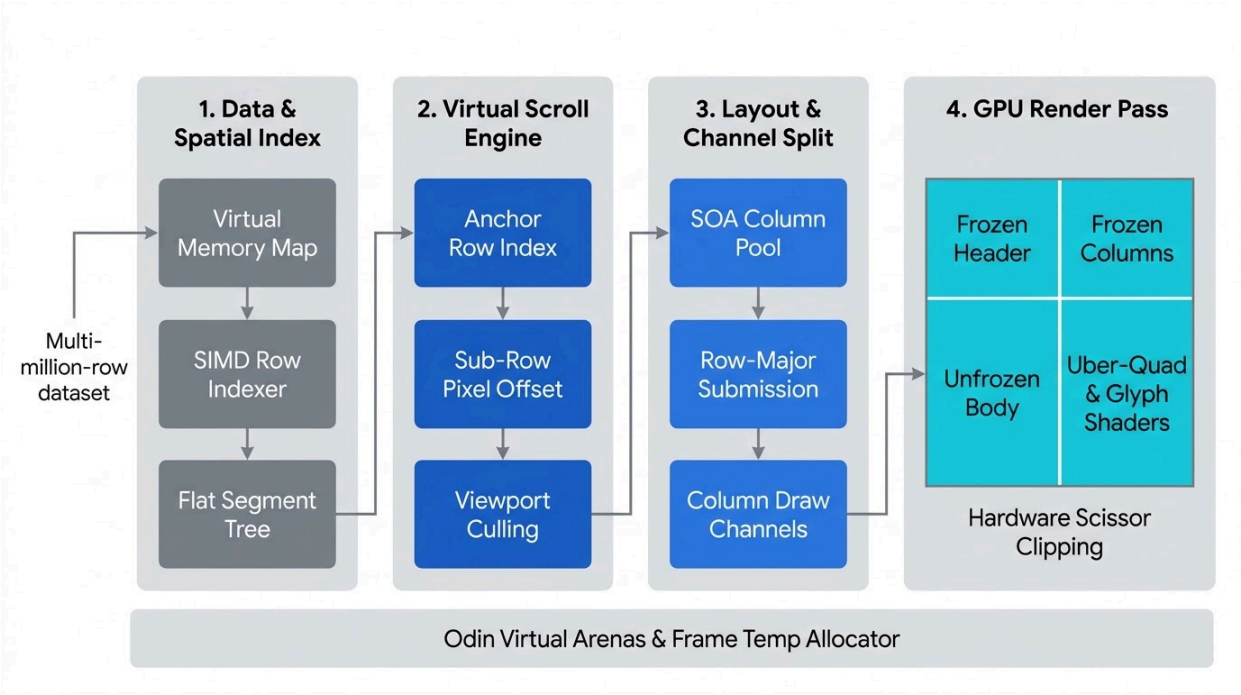
Architecting a High-Performance Native GPU Data Grid and Table Widget in Odin (Bifrost)

Architecting a native GPU table and data grid widget in Odin (bifrost) capable of sustaining 60 to 120 FPS across 10 million to 100 million+ rows requires decoupling layout coordinate spaces, GPU draw-call batching, spatial indexing, and string formatting from total dataset cardinality (N)[1]. Traditional Document Object Model (DOM) and naive immediate-mode UI tables degrade or fail above 150,000 to 1,000,000 rows due to three structural bottlenecks: layout engine thrashing from per-cell node allocation[1], GPU draw-call explosion caused by row-major cell submission conflicting with column-major scissor clipping[1], and IEEE-754 f32 mantissa exhaustion when virtual scroll containers exceed 2^23 pixels (8,388,608 px, or 349,525 rows at 24 px height)[9]. Even heavily optimized web grids with row and column virtualization exhibit severe frame-rate drops at 1 million rows: a benchmark published by 1771 Technologies (creators of LyteNyte Grid) on an Apple M4 MacBook Air showed AG Grid dropping to 27 FPS, MUI X Data Grid to 33 FPS, DevExtreme to 17 FPS, and Handsontable crashing with out-of-memory errors above 150,000 rows (while LyteNyte maintained 60 FPS)[6]. By contrast, engineering blogs and technical documentation for Canvas and WebGL/GPU architectures (such as Glide Data Grid and Quadratic) report a consistent 60 FPS across 10 million+ rows or cells by issuing only 10 to 50 batched GPU draw calls per frame[1], while a developer benchmark moving Fenwick-tree virtualization math into compiled Rust/WASM reported 115 to 120 FPS across 1 to 10 million items[3].

Because no public Odin library or specification named bifrost appears in available documentation, this report treats bifrost as your target native Odin UI module and synthesizes proven production designs from Dear ImGui (imgui_tables), egui_extras::TableBuilder, Clay, Zed's gpui and Rune, LibreOffice Calc's mdds, and Quadratic into idiomatic Odin. To achieve sub-millisecond CPU frame budgets, bifrost integrates five core subsystems: (1) a hybrid immediate-interface / retained-state layout engine backed by Structure-of-Arrays (#soa) buffers and core:mem/virtual arenas[7]; (2) dual-path GPU clipping via CPU draw-channel transposition (ImDrawListSplitter) or instanced shader content masks (gpui/Rune)[7]; (3) f32-safe anchor-offset virtual scrolling (RowScroller) that keeps all rendered coordinates near 0.0 in viewport-local space[4]; (4) adaptive sparse spatial indexing ranging from zero-byte uniform math to a 15.62 KB Run-Length Encoded (RLE) Flat Segment Tree (mdds::flat_segment_tree) or a 19.37 MB Two-Level Chunked Prefix-Sum Array for 10 million rows[10]; and (5) zero-allocation ingestion and cell rendering using virtual.map_file, core:simd (u8x32/u8x64) CSV scanning, stack-buffered fmt.bprintf formatting, and monochrome-compressed glyph vertex buffers[1].



1. Immediate/Retained-Mode Hybrid Table Layout Architecture

Pure retained-mode spreadsheets—where every visible or logical cell is backed by a persistent widget node in a scene graph—incur heavy synchronization, diffing, and memory overhead during rapid scrolling or streaming updates[1]. Conversely, a pure stateless immediate-mode layout cannot resolve table column widths in a single pass when column sizing depends on cell contents (WidthAuto) or proportional remaining space (WidthStretch), nor can it persist user-driven column resizing, reordering, and scroll anchoring without retained backing storage[7].

The Hybrid Immediate-API / Retained-Pool Design

bifrost resolves this tension by adopting the hybrid paradigm pioneered by Dear ImGui's imgui_tables[7] and refined by egui_extras::TableBuilder[20] and Clay[21]. The caller interacts with a zero-retention immediate procedure call sequence per frame—begin_table(), setup_column(), setup_scroll_freeze(), table_headers_row(), a virtualized loop calling table_next_row() and table_next_column(), and end_table()[7]—while bifrost internally retrieves a persistent Table_Retained_State from an internal handle map keyed by Table_ID (with an optional instance_no salt if the same table ID is instantiated multiple times in a single frame, mirroring ImGuiTableInstanceData)[7].

To maximize CPU L1/L2 cache locality during per-frame layout updates (table_update_layout) and column culling, bifrost splits table state into three distinct lifetime tiers using Odin's native #soa (Structure of Arrays) directives and core:mem/virtual arenas[7]:

State Structure	Lifetime & Backing Allocator	Core Responsibilities & Data Fields	Architectural Inspiration
Table_Retained_State	Persistent across frames; allocated in a vmem.Arena (Growing mode) owned by the Bifrost_Context[13].	Stores Table_ID, flags, anchor_row: i64, offset_within_row: f32, scroll_x: f32, freeze_cols: i16, freeze_rows: i16, spatial index pointers, and the persistent #soa[]Column_State slice[4].	ImGuiTable + ImGuiTableSettings[1][1]
#soa[]Column_State	Persistent per table; allocated from the table's retained arena upon first begin_table() initialization[7].	Stores per-column flags, width_request: f32, stretch_weight: f32, width_auto: f32, width_computed: f32, min_x: f32, max_x: f32, clip_rect: [4]f32, display_order: i16, is_visible: bool, and draw_channel: u16[7].	ImGuiTableColumn[7]
Table_Temp_Data	Transient per active table in the layout stack; allocated from a pre-reserved vmem.Arena (Static mode) or context.temp_allocator reset every frame[7].	Stores current_row: i32, current_col: i16, row_bg_color: u32, row_max_height: f32, visible column index list, and the active Draw_Channel_Splitter buffers[7].	ImGuiTableTempData[11]

Using #soa[]Column_State in Odin transforms an array of structs into parallel contiguous arrays

for each field (columns.width_computed[i], columns.min_x[i], columns.is_visible[i])[12]. When a data grid contains 300 to 500 columns (common in enterprise reporting schemas and wide feature-engineering matrices[6]), horizontal viewport culling only touches the min_x, max_x, and flags arrays—fitting 16 column bounds into a single 64-byte cache line instead of pulling 64-to-128-byte AoS column structs into L1 cache.

Column Sizing Policies and the to_lengths / TableUpdateLayout Algorithm

Following TableSetupColumn() in Dear ImGui[7] and Column in egui_extras (crates/egui_extras/src/sizing.rs)[20], bifrost supports four column sizing policies, all of which can be constrained by .at_least(min_w) and .at_most(max_w) bounds[20]. A Width_Exact policy (Column::exact(w)) enforces a strict fixed width w (excluding cell padding, so total column span is w + 2 * cell_padding_x[7]), disables interactive resizing, and clips content[20]. A Width_Fixed policy (Column::initial(w)) initializes the column to w logical points, permits manual dragging via column border separators, and excludes the column when distributing extra container width[7]. A Width_Auto policy (Column::auto()) dynamically sizes the column to the maximum measured content width (width_auto) of cells submitted in that column during the active viewport pass, plus header width[7]. Finally, a Width_Stretch or Remainder policy (Column::remainder(weight)) divides remaining horizontal space in the table's inner_width (or outer_size.x when horizontal scrolling is disabled) proportionally to weight / sum_weights, clamped to [min_w, max_w][7]. When table_update_layout(table) executes—triggered automatically on the first call to table_headers_row() or table_next_row() after begin_table()[7]—it resolves column geometry in four deterministic steps:

1. **Apply Queued Interaction Requests (table_apply_queued_requests):** Process any mouse-drag column resize (table_set_column_width), double-click auto-fit trigger (.auto_size_this_frame), column hide/unhide toggle, or drag-and-drop column reordering (table_set_column_display_order) captured during the *previous* frame's border hit-testing[7]. Processing interactions at the start of table_update_layout guarantees that all rows in the current frame see identical, coherent column coordinates[7].
2. **Resolve Fixed and Auto Widths:** Iterate over all enabled columns in display_order. Sum the widths of Width_Exact, Width_Fixed, and Width_Auto columns (using the retained width_auto measured from the prior frame, or an initial suggestion such as 100.0 points on frame 0[20]), adding 2 * cell_padding_x per column[7].
3. **Distribute Remainder / Stretch Space (to_lengths):** Compute remaining_width = max(0.0, available_inner_width - total_fixed_and_auto_width). Distribute remaining_width across all Width_Stretch columns proportional to stretch_weight, iteratively clamping columns that violate min_w or max_w and redistributing the residual width among unclamped stretch columns[7].
4. **Compute Cumulative X-Offsets and Horizontal Culling:** Walk display_order to assign each column's screen-space min_x and max_x (accounting for freeze_cols: columns with display index < freeze_cols are positioned relative to the table's left outer border, whereas unfrozen columns are shifted by -scroll_x)[2]. Intersect each column's [min_x,

max_x] with its quadrant's horizontal viewport bounds to set column.is_visible and precompute column.clip_rect[7].

Solving the Auto-Sizing Dilemma in Multi-Million-Row Virtualized Grids

A classic architectural trap in immediate-mode tables is handling Width_Auto on the very first frame a table appears, or when the user double-clicks a column divider to auto-fit[20]. Two-pass layout engines like Clay (Clay_CalculateFinalLayout) build a complete node tree in memory, run bottom-up fit-to-children sizing, run top-down grow/shrink distribution, wrap text, and emit a Clay_RenderCommandArray[21]. Similarly, egui_extras::TableBuilder introduced a sizing_pass flag (in v0.28.0 and expanded in v0.29.0) that runs an initial measurement pass on the first frame to calculate required column bounds before finalizing element positions[20].

In a 10-million-row CSV viewer or database grid, however, **you can never run a sizing pass over the entire dataset**—measuring text widths across 10 million rows would stall the UI thread[20]. Instead, bifrost should combine viewport-bounded measurement, frame-0 sample estimation, and monotonic width ratcheting. Just as egui_extras restricts dynamic auto-width calculation in virtualized TableBody::rows or heterogeneous_rows to the currently visible viewport rows[20], bifrost accumulates column.width_auto_curr = max(column.width_auto_curr, cell_measured_width) strictly across the ~50 to 100 visible rows rendered in the current frame[7].

To avoid a 1-frame visual jump on initial open without paying for a double-submission frame pass, bifrost can synchronously measure the column header string plus a deterministic sample of the first 64 rows (using fast glyph advance lookups without emitting GPU geometry) inside begin_table() when table.frame_count == 0. Furthermore, because continuous shrinking and growing of Width_Auto columns during rapid scrolling causes distracting horizontal border jitter, Width_Auto in virtualized mode should monotonically ratchet upward (width_computed = max(width_computed, width_auto_curr)), while double-clicking a column separator sets .auto_size_this_frame = true[20] to snap the column tightly to the current viewport's maximum content width.

Code snippet

```
package bifrost

import vmem "core:mem/virtual"

Column_Flags :: distinct bit_set[enum u8 {
    Width_Fixed,
    Width_Stretch,
    Width_Auto,
    No_Resize,
```

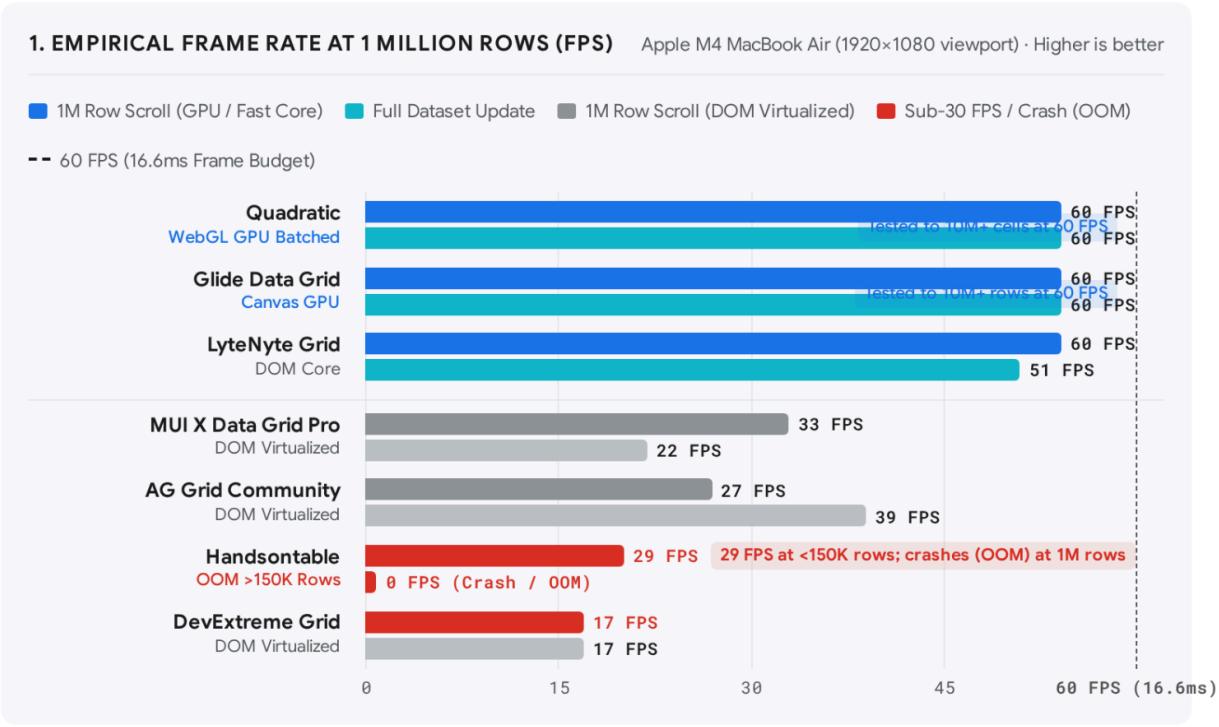


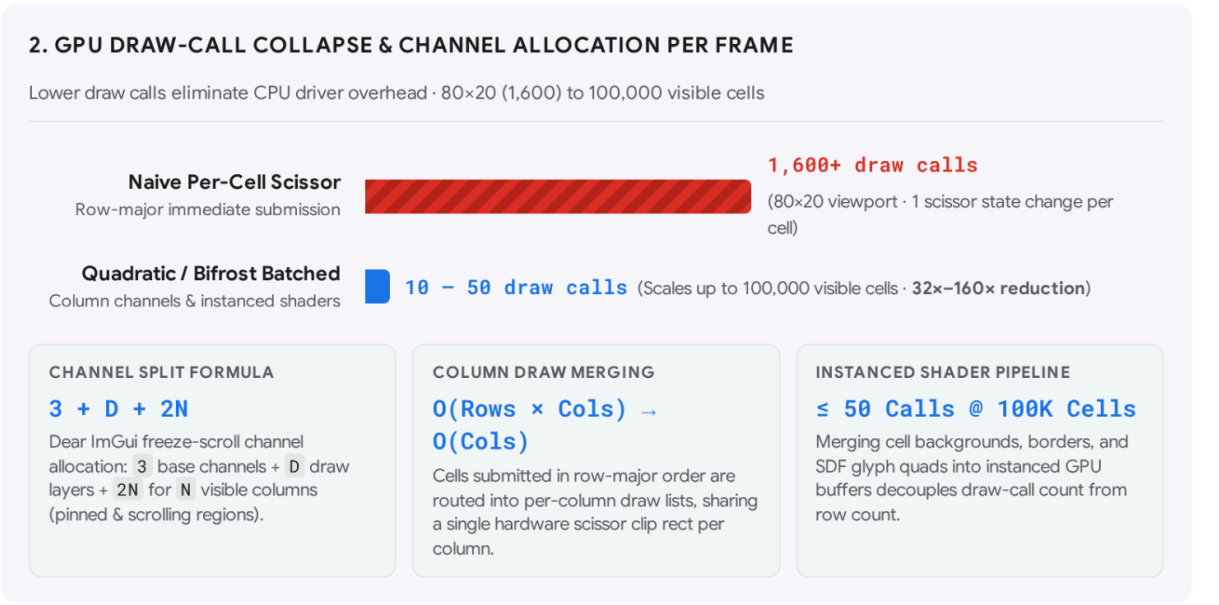
2. Column Draw-Channel GPU Batching and Hardware Scissor Clipping

In any data grid, cell text and custom cell widgets must be clipped to their column's horizontal boundaries ([col.min_x, col.max_x]) so long strings do not bleed into adjacent columns[7]. Simultaneously, callers must be able to submit data row-by-row (for row in visible_rows { for col in visible_cols { ... } }), because database rows, CSV records, and telemetry structs are accessed

row-wise or via row-indexed columnar slices[7].
If a UI renderer naively pushes a hardware scissor rectangle (glScissor, vkCmdSetScissor, or wgpu::RenderPass::set_scissor_rect) every time table_next_column() advances from column c to c + 1, the GPU command buffer cannot batch across columns. For a modest viewport of 80 visible rows × 20 visible columns = 1,600 cells[32], changing the scissor rectangle per cell generates **1,600 separate GPU draw calls per frame** (or 3,200+ if cell backgrounds and text use separate pipelines)[1]. To collapse 1,600 to 100,000 visible cell renders into 10 to 50 draw calls per frame—as reported in Quadratic's WebGL architecture[1]—bifrost can implement either **CPU Draw-Channel Transposition (ImDrawListSplitter)** or **GPU Instanced Uber-Quad & In-Shader Clipping (gpui / Rune)**.

GPU-Batched Grids Sustain **60+ FPS at 1M Rows** While Per-Cell Architectures Degrade





Top: Apple M4 MacBook Air benchmarks (1920×1080 viewport) demonstrate that most DOM-virtualized grids drop to 17–33 FPS at 1 million rows (with Handsontable crashing above 150K rows), whereas Canvas/WebGL GPU-batched engines (Glide, Quadratic) and LyteNyte maintain 60 FPS. **Bottom:** Transposing row-major submissions into column draw channels or instanced shaders collapses 1,600+ cell renders into 10–50 GPU draw calls per frame.

Data sources: Dear ImGui Tables Architecture (imgui_tables.cpp), Industry 1M-Row Data Grid Benchmarks (Quadratic, Glide Data Grid, LyteNyte).

Path A: CPU Draw-Channel Splitting and Scissor Merging

(ImDrawListSplitter Pattern)

In Dear ImGui's imgui_tables.cpp, TableSetupDrawChannels() partitions the active ImDrawList into multiple virtual draw channels before any rows are submitted[7]. Instead of writing vertices and indices into a single monolithic command buffer during row iteration, Draw_Channel_Splitter gives each channel its own lightweight _CmdBuffer and _IdxBuffer slices while sharing a single contiguous vertex buffer (VtxBuffer) so vertex generation never copies vertex data[33].

Channel Topology for 4-Quadrant Frozen Panes

When a table uses pinned/frozen rows (freeze_rows > 0, such as sticky headers or pinned summary rows) and pinned/frozen columns (freeze_cols > 0, such as pinned ID or timestamp columns)[2], the viewport is divided into four distinct hardware scissor quadrants: Top-Left (Frozen Rows × Frozen Columns, static in both axes and clipped to [table_min.x, table_min.y, freeze_x, freeze_y]), Top-Right (Frozen Rows × Unfrozen Columns, scrolling horizontally by -scroll_x and clipped to [freeze_x, table_min.y, table_max.x, freeze_y]), Bottom-Left (Unfrozen Rows × Frozen Columns, scrolling vertically by -offset_within_row and clipped to [table_min.x, freeze_y, freeze_x, table_max.y]), and Bottom-Right (Unfrozen Body, scrolling in both axes and clipped to [freeze_x, freeze_y, table_max.x, table_max.y]).

To prevent frozen cells from being overwritten by scrolling body cells or vice versa, Dear ImGui's TableSetupDrawChannels() allocates 3 + D + 2N channels when `FreezeRows | | FreezeColumnsis active (or2 + D + 2NforFreezeRowsalone, and2 + D + Nwhen no freeze is active, whereNis the number of visible columns andD = 1 accounts for border/background routing) [cite: 7, 25, 34]. Channel 0 (TABLE_DRAW_CHANNEL_BG0) receives unfrozen body row backgrounds such as zebra striping, hover highlights, and selection fills[7]. Channel 1 (TABLE_DRAW_CHANNEL_BG2_FROZEN) receives frozen row/column backgrounds and headers[7]. Channel 2 (TABLE_DRAW_CHANNEL_NOCLIP) receives unclipped foreground decorations when ImGuiTableFlags_NoClipis used[7]. The remaining2Nchannels are split intoNchannels for foreground cell contents of the visible columns in the unfrozen body rows andN` channels for the visible columns in the frozen top rows[7].

During row submission, switching from column c to c + 1 in table_next_column() simply swaps the active channel pointer (splitter_set_current_channel(draw_list, col_channel)), copying the channel's CmdBuffer and IdxBuffer headers into the active draw_list[33]. When table_end_row() detects the boundary between the last frozen row and the first unfrozen row (table.current_row + 1 == table.freeze_rows), it reassigns each column's draw_channel from the Frozen Top channel range to the Unfrozen Body channel range[7].

The TableMergeDrawChannels Scissor-Expansion Optimization

If end_table() merely concatenated all 3 + D + 2N channels in order, the GPU would still execute 2N separate draw calls—one for each column's unique ClipRect—even when 95% of cells contain short numbers or status strings that do not overflow their column width. To eliminate this overhead, Dear ImGui's TableMergeDrawChannels() performs a critical geometric optimization prior to calling ImDrawListSplitter::Merge()[7]:

1. **Track Column Overflow:** As items are submitted into a cell during the frame, bifrost

checks whether the item's bounding box item_max_x actually exceeds the column's right clip boundary col.clip_rect.max_x (or whether the caller passed a guarantee flag that the cell does not overflow).

2. **Expand Non-Clipped Column Rects to Quadrant Bounds:** If a column's contents in a given quadrant never overflowed col.clip_rect, table_merge_draw_channels() expands that channel's ClipRect horizontally toward the limits of the **entire quadrant's bounding rectangle** (e.g., expanding col.clip_rect.min_x and max_x to the full Top-Right or Bottom-Right quadrant limits)[7].
3. **Coalesce Adjacent Index Buffers:** When splitter_merge() iterates through the reordered channels and appends their index buffers together, it compares memcmp(&curr_cmd.clip_rect, &prev_cmd.clip_rect, size_of([4]f32)) and curr_cmd.texture_id == prev_cmd.texture_id[33]. Because all non-overflowing columns in the same quadrant now share the *exact same quadrant ClipRect*, their index buffers merge into a **single contiguous GPU draw command**[8]. Only columns that genuinely overflow their width retain a narrow column-specific scissor rect.

Critical Implementation Pitfall (VtxOffset at >64K Vertices): As documented in Dear ImGui Issue #3129[33], if your renderer uses 16-bit vertex indices (u16) and a dense data grid exceeds 65,536 vertices in a single frame, bumping _VtxCurrentOffset mid-frame corrupts pre-allocated draw commands across split channels unless splitter_set_current_channel() explicitly checks if last_cmd.vtx_offset != draw_list.vtx_current_offset and updates last_cmd.vtx_offset (when ElemCount == 0 && !UserCallback) or spawns a new draw command[33]. In bifrost, either use **32-bit index buffers (u32)** by default or prefer the instanced GPU approach below.

Path B: Instanced "Uber-Quad" & In-Shader ContentMask Clipping (gpui / Rune Pattern)

Modern GPU-native UI frameworks such as Zed's gpui and Rune bypass CPU-side index buffer splitting and hardware glScissor state changes altogether by moving clipping into the shader via an instanced **"Uber-Quad"** pass and a GPU clip table (ContentMask)[14].

Instead of generating 4 vertices and 6 indices per rectangle on the CPU, bifrost binds a single static 4-vertex unit quad ([0,0], [1,0], [0,1], [1,1]) and streams **one compact instance struct per primitive** into a storage or instance buffer[15]. Every instance carries its own clip_bounds: [4]f32 (or a u16 index into a uniform/storage buffer of active column-quadrant clip rectangles, which in a table with N visible columns contains at most 4 + 2N entries)[15]:

Code snippet

```
// 48-byte aligned GPU Instance for Cell Backgrounds, Borders, Selection Boxes, and Glyphs
Bifrost_Quad_Instance :: struct #packed {
    bounds:      [4]f32, // {x, y, width, height} in physical screen pixels
```

```
    uv_bounds:    [4]f16, // Normalized atlas UVs (0,0,0,0 for solid rects)
    color:        u32,    // Packed RGBA8 (Format.U8_RGBA_NORM)
    border_color: u32,    // Packed RGBA8 for cell border / outline
    clip_idx:     u16,    // Index into GPU Clip_Rect_Table SSBO/UBO
    corner_radius: u8,    // Pixel radius for badges/selections
    flags:        u8,     // Bit 0: Solid vs. MSDF Text vs. RGBA Sprite; Bit 1-2: Border mask
}
```

In the vertex or fragment shader, the instance's bounds are intersected against clip_table[instance.clip_idx] via a Signed Distance Field (SDF) box mask[15]. Because the clip rectangle is evaluated per-instance in the shader rather than via pipeline state mutations (set_scissor_rect), **every cell background, border, badge, and glyph across all 4 frozen/unfrozen quadrants and all N columns can be submitted in direct row-major order and drawn in 1 to 4 instanced GPU draw calls per frame**[1].

Crisp 1px Grid Lines and Luminance-Adaptive Contrast

Drawing grid lines as naive 1px primitives often produces blurry 2px gray lines on standard displays or inconsistent thickness on fractional HiDPI (devicePixelRatio = 1.25, 1.5, 2.0) displays because integer coordinates sit on pixel boundaries, causing a 1.0px stroke to expand 0.5px on either side and straddle two adjacent pixels[37]. bifrost handles grid lines cleanly using two complementary techniques. First, for variable-size cells, bifrost snaps logical layout coordinates (x, y) to physical framebuffer pixels (px = round(logical_x * dpi_scale)) and emits exact 1-physical-pixel-wide filled quads (or uses Dear ImGui v1.93's axis-aligned AddLineV(x, y1, y2, col) and AddLineH(x1, x2, y, col) primitives with ImDrawFlags_AALineEnds disabled by default to avoid extra vertex/index overhead across tens of thousands of lines)[41]. Second, for uniform-height telemetry logs or infinite spreadsheet areas, bifrost can render the entire grid of vertical and horizontal lines in a **single full-screen quad** using screen-space partial derivatives (fwidth)[42] and Figma/FigJam's perceptual luminance contrast formula (weights = vec3(0.299, 0.587, 0.114)), which automatically darkens grid lines on light backgrounds (luminance > 0.5) and lightens them on dark backgrounds[43]:

```
OpenGL Shading Language
// Screen-space anti-aliased grid line evaluation (adapted for Bifrost viewport)
vec2 grid_dist = abs(fract((world_coord - cell_origin) / cell_size - 0.5) - 0.5)
               / fwidth(world_coord) * cell_size;
float line_alpha = 1.0 - clamp(min(grid_dist.x, grid_dist.y), 0.0, 1.0);
```

3. Float32-Safe Virtual Scrolling Beyond 2^24 Pixels

A standard virtual scroller calculates the total scrollable height of the dataset as total_height =

num_rows * row_height, positions the scrollbar using scroll_top in [0, total_height - viewport_height], derives start_index = floor(scroll_top / row_height), and translates the rendered rows to start_index * row_height - scroll_top[44]. While this works for 10,000 rows, it fails catastrophically at multi-million-row scales due to both **browser/UI layout coordinate caps** and **IEEE-754 32-bit floating-point (f32) precision exhaustion**[4].

Quantitative Analysis of IEEE-754 f32 Precision Degradation

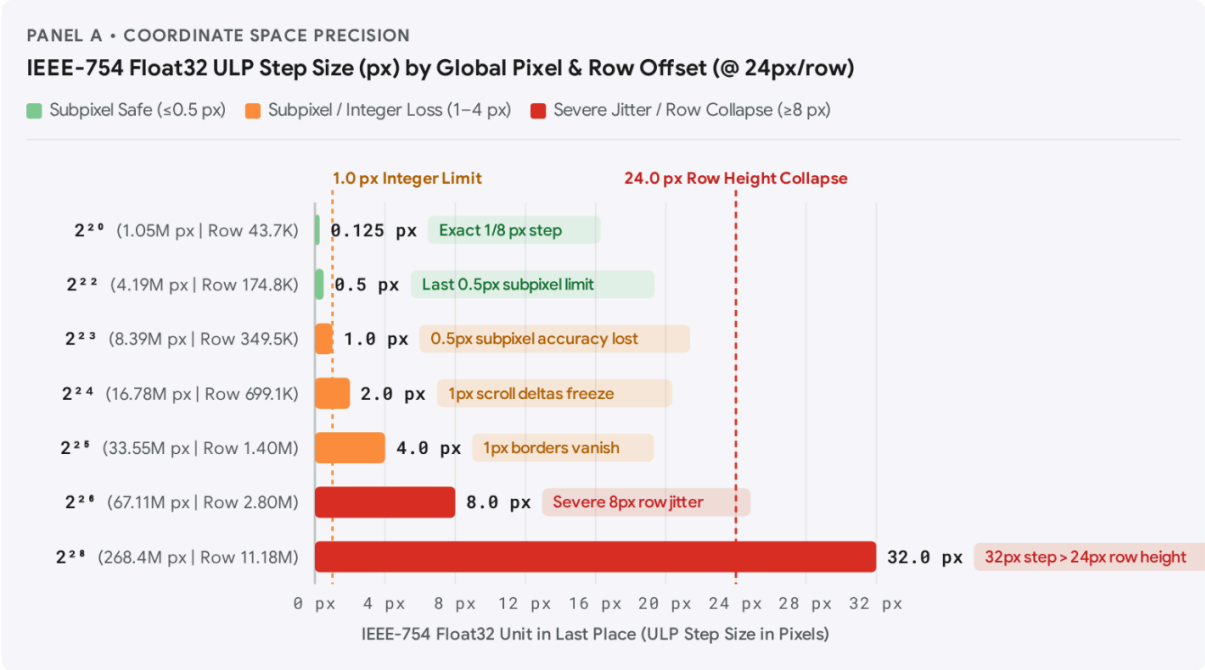
An IEEE-754 f32 has 1 sign bit, 8 exponent bits, and 23 explicit mantissa bits (24 bits of significand precision, including the implicit leading 1). Within any power-of-two interval [2^k, 2^(k+1)), the step size between adjacent representable floats—the Unit in the Last Place (ULP)—is 2^(k-23) pixels[10]. In web browsers, this is compounded by fixed-point layout limits (16,777,216 px scrolling/cull limit and 33,554,428 px element height limit in Chromium; 33,554,428 px in Safari; and 17,895,697 px app-unit limit in Firefox)[9]. Even in a native Odin application with no browser DOM, storing scroll_top or vertex Y-coordinates in f32 causes progressive visual failure as row indices climb[4]:

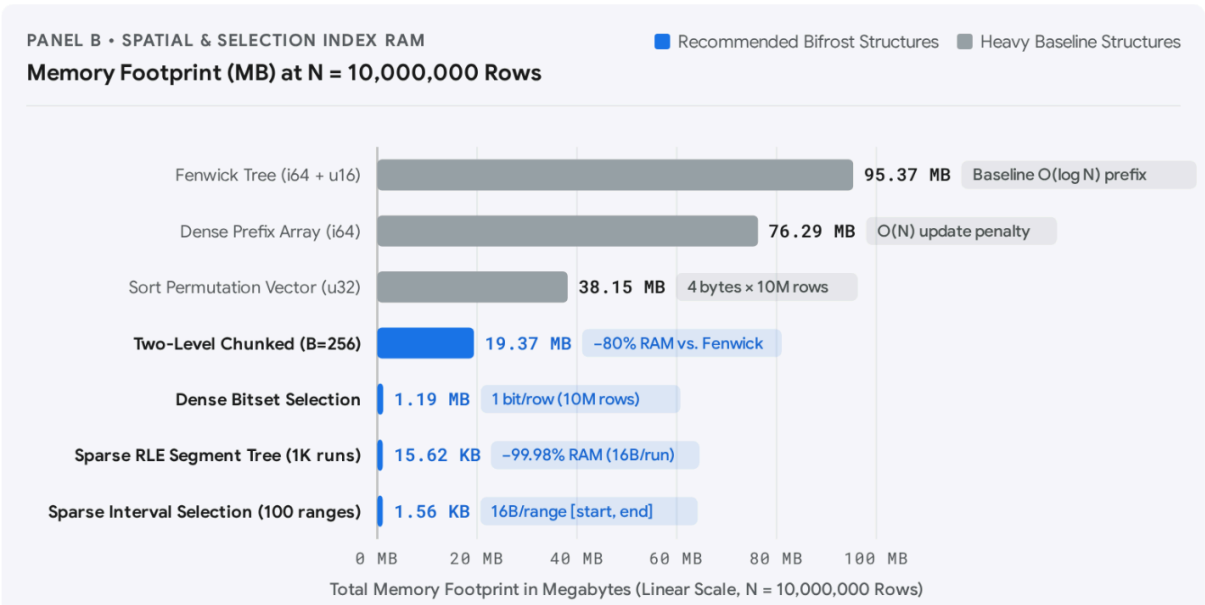
Global Scroll Offset (Pixels)	Binaryade Threshold	f32 ULP Step Size (px)	Row Index (@ 24 px/row)	Visual & Interaction Failure Mode in Naive f32 UI
1,048,576 px	2^20	0.125 px	Row 43,690	Subpixel precision begins coarsening; smooth spring animations lose sub-eighth-pixel fidelity[10].
4,194,304 px	2^22	0.500 px	Row 174,762	Fractional HiDPI (1.25x, 1.5x, 1.75x) physical pixel snapping fails[10].
8,388,608 px	2^23	1.000 px	Row 349,525	**Subpixel 0.5 px lost:** Crisp 0.5 px centered line

				offsets become unrepresentable; borders blur[10].
16,777,216 px	2^24	2.000 px	Row 699,050	**Integer pixel limit exceeded:** Odd pixel coordinates cannot be represented (16,777,216.0 + 1.0 == 16,777,216.0). Slow 1px trackpad scrolling freezes completely[9].
33,554,432 px	2^25	4.000 px	Row 1,398,101	**1px borders vanish or double:** Row heights round to multiples of 4 px; matches Chrome/Safari's 33,554,428 px hard cap[9].
67,108,864 px	2^26	8.000 px	Row 2,796,202	**Severe row jitter:** 24px rows alternate between 24px and 32px heights during scrolling; text baselines jump wildly[10].

268,435,456 px	2^28	32.000 px	Row 11,184,810	**Complete row collapse:** ULP (32 px) exceeds row height (24 px); adjacent rows receive identical Y-coordinates and overwrite each other[10].
1,073,741,824 px	2^30	128.000 px	Row 44,739,242	**Multi-row collapse:** 5 to 6 consecutive rows collapse onto the exact same screen Y-coordinate[10].

IEEE-754 **Float32 Precision Collapses Above 350K Rows** While Sparse Indexes Cut RAM by 99%





Top: As global pixel offset passes 8.39M px (Row 349,525 at 24px/row), float32 loses 0.5px subpixel accuracy; by 268.4M px (Row 11.18M), the 32px float32 step exceeds the 24px row height, collapsing adjacent rows onto the same pixel. Bottom: At 10 million rows, a Two-Level Chunked Array (19.37 MB) cuts Fenwick Tree memory (95.37 MB) by 80%, while a Sparse RLE Flat Segment Tree (15.62 KB for 1,000 runs) requires virtually zero RAM.

Data sources: IEEE-754 Binary32 Hardware Arithmetic & Odin Data Structure Memory Layout Benchmarks (NumPy float32 ULP & byte-alignment calculations).

The RowScroller Anchor-Plus-Sub-Offset Architecture

To make bifrost immune to f32 precision loss across 10 million to 10 billion+ rows without paying the overhead of 64-bit floats (f64) in GPU vertex buffers or UI layout rects, bifrost implements the **Anchor-Plus-Sub-Offset (RowScroller) coordinate model**[4]:

1. **Decoupled Scroll State Representation:** Never store vertical scroll position as a global pixel offset. Instead, represent vertical scroll state as a two-field struct containing anchor_row: i64 and offset_within_row: f32[4], where anchor_row in [0, total_rows - 1] is the zero-based 64-bit integer index of the first visible row intersecting the top edge of the unfrozen body viewport, and offset_within_row in [0.0, height_at(anchor_row)) is the exact f32 number of pixels of anchor_row that have scrolled past the top of the body viewport[4].
2. **Viewport-Local f32 Layout Coordinates:** When laying out visible rows for the current frame, bifrost starts at row_idx = table.anchor_row with a viewport-local Y-coordinate of y_local = body_top_y - table.offset_within_row[4]. For example, if the user is at anchor_row = 9_000_000 and offset_within_row = 24.0 inside an 80.0 px tall row, row 9,000,000 is placed at body_top_y - 24.0, and row 9,000,001 starts at body_top_y + 56.0[4]. Every visible row's screen coordinate remains within [0.0, window_height] (typically under 2160.0 px), where f32 maintains sub-micrometer precision (0.000122 px ULP).
3. **Overscan Walk Backward & Forward:** To support smooth scrolling or async cell rendering with overscan_px (e.g., 200.0 to 400.0 px or 3 to 10 items[4]), the layout routine walks backward from anchor_row (decrementing idx and subtracting height_at(idx) from y_local until y_local <= body_top_y - overscan_px or idx == 0) and then walks forward accumulating height_at(idx) until y_local >= body_bottom_y + overscan_px or idx == total_rows[4].
4. **Local Delta Consumption (move_by):** When the user scrolls via mouse wheel, touchpad momentum, or keyboard arrows by delta_y: f32 pixels, scroll_move_by(&table, delta_y) adds delta_y directly to offset_within_row and normalizes across row boundaries without referencing a global pixel offset[4]:

Code snippet

```
scroll_move_by :: proc(
    t:          ^Table_Retained_State,
    total_rows: i64,
    delta_y:    f32,
    viewport_h: f32,
    height_at:  proc(user_data: rawptr, row: i64) -> f32,
    user_data:  rawptr,
)
```

```
    if total_rows <= 0 do return
    t.offset_within_row += delta_y

    // Scrolling downward: advance anchor_row while offset exceeds current row height
    for t.anchor_row < total_rows - 1 {
        row_h := height_at(user_data, t.anchor_row)
        if t.offset_within_row < row_h do break
        t.offset_within_row -= row_h
        t.anchor_row += 1
    }

    // Scrolling upward: step anchor_row backward while offset is negative
    for t.offset_within_row < 0.0 && t.anchor_row > 0 {
        t.anchor_row -= 1
        row_h := height_at(user_data, t.anchor_row)
        t.offset_within_row += row_h
    }

    // Clamp at top boundary
    if t.anchor_row == 0 && t.offset_within_row < 0.0 {
        t.offset_within_row = 0.0
    }

    // Clamp at bottom boundary so the viewport never overscrolls past the final row
    scroll_clamp_to_end(t, total_rows, viewport_h, height_at, user_data)
}

scroll_clamp_to_end :: proc(
    t:           ^Table_Retained_State,
    total_rows: i64,
    viewport_h: f32,
    height_at: proc(user_data: rawptr, row: i64) -> f32,
    user_data: rawptr,
) {
    // Verify if remaining content from anchor_row to end fills viewport_h
    rem_h := -t.offset_within_row
    idx := t.anchor_row
    for idx < total_rows && rem_h < viewport_h {
        rem_h += height_at(user_data, idx)
        idx += 1
    }
    if rem_h >= viewport_h do return
```

```
    // Walk backward from the final row to find the maximum valid anchor_row + offset
    end_idx := total_rows -
    accum_h := height_at(user_data, end_idx)
    for end_idx > 0 && accum_h < viewport_h {
        end_idx -=
        accum_h += height_at(user_data, end_idx)
    }
    t.anchor_row = end_idx
    t.offset_within_row = max(0.0, accum_h - viewport_h)
}
```

Dual-Mode Scrollbar Mechanics and the "Infinite Pixels" Downscale Problem

While scroll_move_by handles local wheel and trackpad deltas with subpixel accuracy at any row depth, the **scrollbar thumb** must map global progress across the entire table[45]. When a table has 100 million or 10 billion rows on a 1,000 px scrollbar track, a 1 px mouse drag of the scrollbar thumb corresponds to millions of rows (or 2,189,781,021 virtual pixels / 72,992,701 rows per 1 px scroll move in the 10-billion-row hightable downscaling analysis)[45]. If wheel events and scrollbar thumb drags both mutated a single downscaled pixel coordinate, fine scrolling would become impossible and the vast majority of rows would sit in unreachable gaps between 1 px scroll steps[45].

bifrost solves this by separating scrollbar thumb interaction from delta scrolling[4]. For scrollbar rendering, bifrost computes thumb height (thumb_h = max(16.0, track_h * (f64(viewport_h) / f64(total_content_h)))) and normalized track progress u in [0.0, 1.0] using 64-bit floating-point arithmetic (f64), either from the 64-bit cumulative pixel offset ((f64(prefix_sum(anchor_row)) + f64(offset_within_row)) / f64(max_scroll_px)) or directly from the row index ratio (f64(anchor_row) / f64(max(1, total_rows - visible_rows)))[4]. Only when the user actively drags the scrollbar thumb (is_dragging_thumb == true) does bifrost invoke jump_to(index), converting the normalized thumb ratio u back into a target anchor_row = i64(u * f64(total_rows - visible_rows)) and resetting offset_within_row = 0.0 (or via O(log N) inverse prefix-sum lookup when variable heights are globally indexed)[4]. As soon as the user releases the thumb and uses the mouse wheel, trackpad, or keyboard, scroll_move_by resumes stepping locally from that exact anchor_row with f32 subpixel precision, ensuring every single row in a multi-million- or multi-billion-row dataset is reachable[4].

4. Sparse Row/Column Spatial Indexing & Multi-Million Selection Storage

To support variable row heights (e.g., expanded log payloads, multi-line text, or user-resized rows), hidden/collapsed rows, and variable column widths without sacrificing 60 to 120 FPS frame times, bifrost must choose a spatial index whose memory and time complexity match the data's sparsity[10].

Quantitative Comparison of Spatial Indexing Data Structures

The table below evaluates the six primary 1D spatial indexing structures across N = 1,048,576 (2^20, Microsoft Excel's fixed row limit[10]) and N = 10,000,000 rows[10]:

Spatial Index Architecture	Memory @ 1M Rows (2^20)	Memory @ 10M Rows (10^7)	Point / Range Update	Offset Lookup (row -> y)	Inverse Lookup (y -> row)	Optimal bifrost Workload
1. Uniform Arithmetic	0 B[10]	0 B[10]	O(1) global	O(1) (row * h)[47]	O(1) (floor(y / h))[44]	High-frequency telemetry logs & fixed-height CSVs[32].
2. Local Window Map (RowScroller)	< 4 KB (window only)[4]	< 4 KB (window only)[4]	O(1) local hash	O(V) local walk (V ≈ 80)[4]	O(1) from anchor_row[4]	10M+ dynamic-height rows where global pixel offset is not needed[4].
3. Sparse RLE Flat Segment Tree (mdds)	7.81 KB (K = 500 runs)[10]	15.62 KB (K = 1,000 runs)[10]	O(K) array shift (K << N)	O(log K) or O(1) w/ hint[50]	O(log K) binary search[51]	Spreadsheets & DB grids with default heights + sparse user resizes/hides[16].
4. Two-Level	2.02 MB (u32 blk + u16)[10]	19.37 MB (i64 blk + u16)[10]	O(N/B) SIMD block add	O(1) blk + O(B) SIMD	O(log(N/B) + log B)	Dense pre-measured

Chunked Array (B = 256)				sum		variable row heights up to 10M rows[10].
5. Fenwick Tree / BIT	6.00 MB (u32 + u16)[10]	95.37 MB (i64 + u16)[10]	O(log N) (idx += idx & -idx)[52]	O(log N) (idx -= idx & -idx)[52]	O(log N) binary lifting[3]	Frequent random single-row height mutations requiring exact global pixel offsets[3].
6. Dense Prefix-Sum Array	4.00 MB (u32) / 8.00 MB (f64)[10]	76.29 MB (i64)[10]	O(N) suffix rebuild[52]	O(1) (prefix[row])[47]	O(log N) binary search[53]	Static variable-size lists (< 500K rows) that rarely mutate after load[53].

1. Sparse RLE Flat Segment Tree (mdds::flat_segment_tree Pattern)

In most CSV viewers and database grids, nearly all 10,000,000 rows share a default height (e.g., 24 px), except for a few hundred or thousand rows that the user manually resized, expanded to inspect JSON payloads, or hid via filtering. Allocating a 95.37 MB Fenwick tree[10] for 10 million rows when only 500 rows differ from 24 px wastes memory and cache bandwidth.

Instead, LibreOffice Calc uses mdds::flat_segment_tree to store row heights, column widths, and row/column visibility across 1,048,576 rows[16]. In bifrost, we represent this as a sorted, contiguous array of 16-byte Run-Length Encoded (RLE) segment descriptors[10]:

```
// 16-byte aligned RLE spatial segment node (1,000 runs = 15.62 KB)
Flat_Segment_Node :: struct {
    start_row:      i64, // Inclusive start index of this uniform-height run
    cum_pixel_offset: i64, // Exact cumulative pixel Y at start_row
    row_height:     u16, // Height in px for rows in [start_row, next_node.start_row) (0 = hidden)
    flags:          u16, // Visibility / collapsed group bitflags
    _pad:           u32,
}
```

Because each node stores both start_row and cum_pixel_offset, finding the exact pixel offset of row r (node.cum_pixel_offset + (r - node.start_row) * i64(node.row_height)) or the exact row at pixel y requires a binary search over only K nodes (O(log K)), which for K = 1,000 is at most 10 comparisons inside 15.62 KB of L1-resident memory[10]. Furthermore, by returning a position_hint: int (the index of the last accessed segment node, just as mdds does[50]), sequential row iteration during viewport layout runs in **O(1) amortized time per visible row**. Setting row_height = 0 on a segment hides millions of contiguous rows without reallocating data[16].

2. Two-Level Chunked Prefix-Sum Array vs. Fenwick Tree

When every single row has a distinct height (e.g., multi-line wrapped chat or log entries) and the application requires exact global pixel coordinates, a 1-indexed Fenwick Tree (BIT) provides O(log N) point updates (for i := idx; i <= n; i += i & -i) and O(log N) prefix queries (for i := idx; i > 0; i -= i & -i)[3]. Moving Fenwick tree virtualization math into compiled Rust/WASM was reported in a developer benchmark to raise 10-million-row scrolling from a browser crash in react-window to ~115 FPS with ~0.1 ms WASM call overhead[3].

In native Odin, however, a Fenwick tree over 10 million rows requires an (N + 1) * 8 byte i64 tree array plus an N * 2 byte u16 raw height array = **95.37 MB**[10], and its bit-strided jumps (idx -= idx & -idx) cause CPU cache misses at large indices. A **Two-Level Chunked Prefix-Sum Array** with block size B = 256 stores one i64 cumulative offset per 256 rows (39,063 blocks = 312.5 KB) alongside the u16 raw height array (20.0 MB), totaling **19.37 MB** (an 80% memory reduction over the Fenwick tree)[10]. Looking up a row's global offset reads block_prefix[row >> 8] and sums at most 255 contiguous u16 values in 16-lane SIMD instructions (simd.u16x16), executing entirely within four contiguous 64-byte CPU cache lines.

3. 2D Spatial Indexing for Merged Cells and Dirty Region Caching

When bifrost needs spreadsheet-style merged cells (row_span > 1 or col_span > 1[2]), storing span metadata inside every cell breaks O(1) viewport culling whenever a merged cell's top-left anchor (r_0, c_0) has scrolled off-screen above or to the left of the viewport while its bottom-right corner (r_1, c_1) still intersects the visible area. Storing merged cell bounding rectangles [r_0, c_0, r_1, c_1] in a flat **R-Tree** (minimum bounding rectangle spatial index[56]) or sorted 2D interval list allows bifrost to query active spans once per frame against the visible

[row_min, col_min, row_max, col_max] window. Similarly, if bifrost operates in a zoomable infinite-sheet mode (like Quadratic, which supports 0.01x to 10x zoom[1]), partitioning the grid into spatial hash buckets of **15 columns × 30 rows per hash** ensures that editing a single cell recalculates vertex geometry only for the 450 cells in that bucket (O(cells per hash) instead of O(visible cells), a 100x+ work reduction), with distant hashes evicted via an LRU cache if memory exceeds a 500 MB threshold[1].

Multi-Million-Row Range Selection and Zero-Copy Sorting/Filtering

In Dear ImGui, ImGuiSelectionBasicStorage stores selected item IDs in a flat sorted vector or set[59]. While suitable for a few thousand items, if a user in a 10-million-row CSV viewer clicks Row 0 and Shift+Clicks Row 9,999,999 (or presses Ctrl+A), inserting 10 million 64-bit IDs into a dynamic array or hash table allocates 80+ MB on the UI thread and freezes the application. bifrost avoids selection stalls by combining a **Dual-Representation Selection Model** with an **Indirect Permutation View**. By default, row selection is stored as a sorted [dynamic][2]i64 array of inclusive [start_row, end_row] disjoint intervals (plus an invert_all: bool flag for O(1) Ctrl+A / Select All). Selecting rows 0..=9_999_999 via Shift+Click inserts a single 16-byte interval {0, 9_999_999}; even 100 separate multi-row range selections consume just **1.56 KB** (100 * 16 B)[10], and checking whether a visible row is selected takes O(log I) via binary search. If a user or filter script programmatically selects thousands of non-contiguous individual rows such that the interval count I exceeds 8,192 intervals (128 KB), bifrost automatically promotes the selection storage to a dense bitset ([]u64 of length ceil(N / 64), or core:container/bit_array[18]). Even for 10,000,000 rows, a dense bitset occupies only **1.19 MB** (1,250,000 bytes)[10] and supports O(1) bitwise lookup ((bitset[row >> 6] & (1 << (row & 63))) != 0). Similarly, sorting or filtering a multi-million-row table never mutates the underlying CSV or database buffer in place; instead, bifrost maintains an optional view_indices: []u32 permutation vector (**38.15 MB** for 10M rows[10], allocated in a dedicated vmem.Arena) mapping visual_row to phys_row = view_indices[visual_row], which remains nil (0 B) when unsorted.

5. Low-Overhead Data Ingestion and Text Cell Rendering Pipelines

Even with O(1) virtual scrolling and 10-draw-call GPU batching, a data grid will drop frames if reading cell data, parsing CSV fields, formatting numbers into strings, or generating glyph quads allocates heap memory or stalls on cache misses across the ~1,600 visible cells every 8.3 ms (120 Hz)[32].

Zero-Copy Memory-Mapped Ingestion (core:mem/virtual) and core:simd Indexing

Unlike web spreadsheets—where Google Sheets enforces a 100 MB file import ceiling and a 20-million-cell workbook pool across all tabs[29]—a native Odin application can open a multi-gigabyte CSV or binary telemetry file instantaneously with minimal heap footprint by mapping the file directly into the process's virtual address space using vmem.map_file_from_path(filename, {.Read})[13].

Rather than parsing every cell of a 10-million-row CSV on load (which would allocate hundreds

of millions of string slices), bifrost's background ingestion worker uses a **two-tier lazy indexing strategy**. In Tier 1, a background thread scans the memory-mapped []u8 slice at 32 or 64 bytes per iteration using Odin's core:simd package (simd.u8x32 on AVX2 or simd.u8x64 on AVX-512) with simd.from_slice, simd.lanes_eq, and simd.extract_msbs[18] to locate newline characters (\n) while tracking RFC-4180 quote parity (") via bitwise XOR (simd.bit_xor)[17]. Instead of storing an i64 byte offset for every single row (**76.29 MB** for 10M rows[10]), Tier 1 records a **sparse checkpoint index** of one u64 byte offset every 64 rows (156,250 * 8 B = 1.25 MB for 10M rows). In Tier 2, during the virtualized render loop, bifrost only tokenizes the ~50 to 80 rows currently visible in [first_visible_row, last_visible_row][2]: it looks up the nearest 64-row checkpoint checkpoints[row >> 6], scans forward at most 63 newlines in L1 cache, and slices the comma-separated fields as zero-copy string views (transmute(string)mapped_bytes[start:end]) pointing directly into the memory-mapped file.

Code snippet

```
package bifrost

import "base:intrinsics"
import "core:simd"
import vmem "core:mem/virtual"

// Vectorized 32-byte-per-step CSV newline & quote scanner using Odin core:simd
build_csv_sparse_row_index :: proc(
    mapped_data: []u8,
    arena:       ^vmem.Arena,
) -> (checkpoints: [dynamic]u64, total_rows: i64) {
    alloc := vmem.arena_allocator(arena)
    checkpoints = make([dynamic]u64, 0, 16384, alloc)
    append(&checkpoints, 0) // Row 0 starts at byte 0

    nl_vec    := simd.u8x32('\n')
    quote_vec := simd.u8x32('"')

    in_quotes: u32 = 0 // 0 = outside quotes, ~0 = inside quotes
    row_count: i64 = 0
    i: int = 0
    n := len(mapped_data)

    for ; i + 32 <= n; i += 32 {
        chunk := simd.from_slice(simd.u8x32, mapped_data[i : i+32])
```

```
        q_mask := transmute(u32)simd.extract_msbs(simd.lanes_eq(chunk, quote_vec))
        nl_mask := transmute(u32)simd.extract_msbs(simd.lanes_eq(chunk, nl_vec))

        // Fast path: no quotes in this 32-byte block
        if q_mask == 0 && in_quotes == 0 {
            for nl_mask != 0 {
                bit_pos := intrinsics.count_trailing_zeros(nl_mask)
                row_count += 1
                if (row_count & 63) == 0 {
                    append(&checkpoints, u64(i + int(bit_pos) + 1))
                }
                nl_mask &= nl_mask - 1 // Clear lowest set bit
            }
            continue
        }

        // Prefix-XOR quote mask to mask out newlines inside quoted fields
        q_run := q_mask
        q_run ~= q_run << 1
        q_run ~= q_run << 2
        q_run ~= q_run << 4
        q_run ~= q_run << 8
        q_run ~= q_run << 16
        q_run ~= in_quotes
        in_quotes = u32(i32(q_run) >> 31) // Propagate sign bit (MSB) to next block

        valid_nl := nl_mask &~ q_run
        for valid_nl != 0 {
            bit_pos := intrinsics.count_trailing_zeros(valid_nl)
            row_count += 1
            if (row_count & 63) == 0 {
                append(&checkpoints, u64(i + int(bit_pos) + 1))
            }
            valid_nl &= valid_nl - 1
        }
    }

    // Scalar tail loop for remaining < 32 bytes
    for ; i < n; i += 1 {
        b := mapped_data[i]
        if b == '"' {
            in_quotes = ~in_quotes
        } else if b == '\n' && in_quotes == 0 {
```

```
        row_count += 1
        if (row_count & 63) == 0 && i + 1 < n {
            append(&checkpoints, u64(i + 1))
        }
    }
    return checkpoints, row_count
}
```

Zero-Allocation Cell Formatting and Fast Text Rendering

For typed database result sets and telemetry structs (where cells contain raw i64, f64, timestamps, or enums rather than pre-formatted strings), calling a heap-allocating formatter per visible cell would trigger 1,600 allocations * 120 FPS = 192,000 heap allocations per second[32]. Instead, bifrost's typed cell renderers (table_cell_int, table_cell_f64, table_cell_timestamp) format directly into a **128-byte stack buffer** using fmt.bprintf from core:fmt[19] and immediately pass the resulting string slice to the glyph layout engine before the stack frame returns:

Code snippet

```
table_cell_f64 :: proc(t: ^Table_Context, val: f64, precision: int = 4) {
    if !table_next_column(t) do return // Skip formatting entirely if column is culled!
    buf: [128]u8
    str := fmt.bprintf(buf[:], "%.*f", precision, val)
    table_draw_cell_text_clipped(t, str)
}
```

If a custom cell renderer requires dynamic scratch allocations that outlive the immediate procedure call until end_table(), it allocates from context.temp_allocator (or a scoped vmem.arena_temp_begin / vmem.arena_temp_end watermark on the UI frame arena)[13].

To minimize CPU-to-GPU vertex bandwidth and avoid frame stalls during text rendering across 1,600 to 100,000 visible cells, bifrost incorporates three text pipeline optimizations drawn from Quadratic, Zed (gpui), and Rune. First, because tabular text inside a data grid is predominantly single-line (white-space: nowrap[62]), table_draw_cell_text_clipped walks the UTF-8 runes of str, advances pen_x by each glyph's horizontal advance width, and **terminates the loop the instant pen_x > col.clip_rect.max_x**. Even if a cell contains a 50,000-character string (Google Sheets' per-cell character limit[29]) inside a 120 px wide column, bifrost only shapes and emits vertices for the first ~15 visible characters, bounding work to O(column_width_in_glyphs).

Second, following Quadratic's shader split[1], bifrost separates text rendering into a **Non-Tinted Text Shader** (for standard monochrome theme text, passing the text color via a uniform) and a

Tinted Text Shader (for colored status cells, links, and syntax-highlighted logs); omitting the 4 per-vertex float color values (4 floats * 4 vertices per glyph) reduces glyph vertex buffer size by **~33%**[1]. Third, bifrost pre-packs ASCII and common Unicode glyphs into a Multi-channel Signed Distance Field (MSDF) font atlas (42px base atlas with dynamic screen-space uFWidth = distanceFieldRange * fontScale * screenScale) for crisp rendering across zoom levels from 0.01x to 10x[1], paired with a GPUI/Rune-style **two-tier line layout cache** so complex Unicode shaping never stalls the first frame[15].

High-Frequency Telemetry Streaming Architecture

For live financial order books, CAN-bus captures, or system telemetry logs streaming thousands of updates per second with sub-16ms refresh cycles[2], bifrost decouples the background ingestion thread from the main render thread using a reserved static virtual memory arena (vmem.arena_init_static, which reserves a large contiguous virtual address range up front and commits physical pages on demand via vmem.commit)[13] and an atomic row counter. Because a Static vmem.Arena never reallocates or moves its base pointer (_base) as physical pages are committed[13], the ingestion thread can append incoming telemetry structs at the tail while the UI render thread concurrently reads visible slices without locks or pointer invalidation. Furthermore, following Quadratic's targeted viewport update model[1], when background records arrive or cells mutate, bifrost checks whether the modified row range intersects the active [first_visible_row, last_visible_row] viewport; if the user is inspecting historical logs at Row 500,000 while new telemetry appends at Row 12,000,000, no cell vertex buffers are regenerated[1]. When the user scrolls to the bottom of a live stream, bifrost engages stick_to_bottom = true[20] to automatically advance anchor_row with new arrivals, disengaging it immediately when a negative wheel delta (delta_y < 0.0) is detected.

6. Reference Implementation Blueprint for bifrost in Odin

The following self-contained architectural skeleton ties together the retained/immediate state lifecycle, TableUpdateLayout sizing pass, RowScroller f32-safe virtual scrolling, and column culling in idiomatic Odin:

Code snippet

```
package bifrost

import "core:fmt"
import "core:math"
import vmem "core:mem/virtual"

Table_Context :: struct {
```

```
    retained:      ^Table_Retained_State,
    outer_rect:    [4]f32, // {min_x, min_y, max_x, max_y}
    body_rect:     [4]f32, // Unfrozen body viewport rect
    freeze_split_x:  f32,  // X boundary between frozen and unfrozen columns
    freeze_split_y:  f32,  // Y boundary between frozen and unfrozen rows
    current_row:   i64,
    current_row_y:  f32,   // Viewport-local f32 Y coordinate
    current_row_h:  f32,
    current_col_idx: i16,  // Index into visible_cols
    is_frozen_row:  bool,
}

// 1. Begin Table & Resolve Column Layout
begin_table :: proc(
    state:      ^Table_Retained_State,
    outer_rect: [4]f32,
    total_rows: i64,
    wheel_dy:   f32,
    wheel_dx:   f32,
) -> (ctx: Table_Context, visible: bool) {
    outer_w := outer_rect[2] - outer_rect[0]
    outer_h := outer_rect[3] - outer_rect[1]
    if outer_w <= 0.0 || outer_h <= 0.0 do return {}, false

    ctx.retained = state
    ctx.outer_rect = outer_rect
    ctx.freeze_split_y = outer_rect[1] + state.header_height + f32(state.freeze_rows) *
state.default_row_h
    ctx.body_rect = {outer_rect[0], ctx.freeze_split_y, outer_rect[2], outer_rect[3]}

    // Update horizontal scroll and float32-safe vertical anchor-offset scroll
    state.scroll_x = max(0.0, state.scroll_x + wheel_dx)
    if wheel_dy != 0.0 {
        body_h := ctx.body_rect[3] - ctx.body_rect[1]
        scroll_move_by(state, total_rows, wheel_dy, body_h, default_height_proc, state)
    }

    // Resolve column widths (Fixed, Auto, Stretch) and horizontal visibility culling
    table_update_layout(&ctx)
    state.frame_count += 1
    return ctx, true
}
```

```
default_height_proc :: proc(user_data: rawptr, row: i64) -> f32 {
    s := cast(^Table_Retained_State)user_data
    return s.default_row_h
}

// 2. Column Width Resolution & Quadrant Scissor Assignment
table_update_layout :: proc(ctx: ^Table_Context) {
    s := ctx.retained
    n_cols := len(s.columns)
    outer_w := ctx.outer_rect[2] - ctx.outer_rect[0]

    fixed_sum:  f32 = 0.0
    weight_sum: f32 = 0.0

    for i in 0 ..< n_cols {
        flags := s.columns[i].flags
        if .Hidden in flags do continue
        if .Width_Stretch in flags {
            w := s.columns[i].stretch_weight
            weight_sum += w > 0.0 ? w : 1.0
        } else if .Width_Auto in flags {
            s.columns[i].width_computed = clamp(
                s.columns[i].width_auto,
                s.columns[i].min_width,
                s.columns[i].max_width,
            )
            fixed_sum += s.columns[i].width_computed
        } else {
            s.columns[i].width_computed = clamp(
                s.columns[i].width_request,
                s.columns[i].min_width,
                s.columns[i].max_width,
            )
            fixed_sum += s.columns[i].width_computed
        }
    }

    rem_w := max(0.0, outer_w - fixed_sum)
    if weight_sum > 0.0 {
        for i in 0 ..< n_cols {
            if .Width_Stretch in s.columns[i].flags && .Hidden not_in s.columns[i].flags {
                w := s.columns[i].stretch_weight > 0.0 ? s.columns[i].stretch_weight : 1.0
                s.columns[i].width_computed = clamp(
```

```
                rem_w * (w / weight_sum)
                s.columns[i].min_width
                s.columns[i].max_width
            }
        }
    }

    // Assign screen-space X coordinates and cull off-screen columns
    cursor_x := ctx.outer_rect[0]
    ctx.freeze_split_x = cursor_x
    vis_list := make([dynamic]i16, 0, n_cols, context.temp_allocator)

    for order_idx in 0 ..< n_cols {
        col_idx := s.columns[order_idx].display_order
        if .Hidden in s.columns[col_idx].flags {
            s.columns[col_idx].is_visible = false
            continue
        }
        is_frozen_col := order_idx < int(s.freeze_cols)
        col_w := s.columns[col_idx].width_computed

        min_x := is_frozen_col ? cursor_x : (cursor_x - s.scroll_x)
        max_x := min_x + col_w
        cursor_x += col_w
        if is_frozen_col {
            ctx.freeze_split_x = cursor_x
        }

        // Horizontal viewport intersection check against frozen/unfrozen quadrant
        quad_min_x := is_frozen_col ? ctx.outer_rect[0] : ctx.freeze_split_x
        quad_max_x := is_frozen_col ? ctx.freeze_split_x : ctx.outer_rect[2]

        clip_min_x := max(min_x, quad_min_x)
        clip_max_x := min(max_x, quad_max_x)

        if clip_max_x > clip_min_x {
            s.columns[col_idx].is_visible = true
            s.columns[col_idx].min_x_eff = min_x
            s.columns[col_idx].max_x_eff = max_x
            s.columns[col_idx].clip_rect = {clip_min_x, ctx.body_rect[1], clip_max_x,
ctx.body_rect[3]}
            append(&vis_list, col_idx)
```

```
        } else {
            s.columns[col_idx].is_visible = false
        }
    }
    s.visible_cols = vis_list[:]
}

// 3. Float32-Safe Visible Row Iterator
Visible_Row_Iterator :: struct {
    ctx:        ^Table_Context,
    total_rows: i64,
    next_row:   i64,
    cursor_y:   f32, // Always remains in local viewport coordinates [body_top - row_h, body_bottom]
}

make_visible_row_iterator :: proc(ctx: ^Table_Context, total_rows: i64) -> Visible_Row_Iterator {
    return Visible_Row_Iterator{
        ctx        = ctx,
        total_rows = total_rows,
        next_row   = ctx.retained.anchor_row,
        cursor_y   = ctx.body_rect[1] - ctx.retained.offset_within_row,
    }
}

step_visible_row :: proc(it: ^Visible_Row_Iterator) -> (row_idx: i64, local_y: f32, ok: bool) {
    if it.next_row >= it.total_rows || it.cursor_y >= it.ctx.body_rect[3] {
        return 0, 0.0, false
    }
    row_idx = it.next_row
    local_y = math.round(it.cursor_y) // Snap to physical pixel grid
    row_h   := it.ctx.retained.default_row_h

    it.ctx.current_row_idx = row_idx
    it.ctx.current_row_y   = local_y
    it.ctx.current_row_h   = row_h
    it.ctx.current_col_idx = -1

    it.cursor_y += row_h
    it.next_row += 1
    return row_idx, local_y, true
}
```

Architectural Summary Checklist for Implementation

1. **Memory & Arenas:** Allocate persistent per-table state in a growing vmem.Arena, raw streaming telemetry in a large reserved Static vmem.Arena, and per-frame visible column lists and channel command buffers in context.temp_allocator[13].
2. **Coordinate Safety:** Never multiply row_index * row_height into an f32. Keep all scroll offsets in (anchor_row: i64, offset_within_row: f32) and compute scrollbar thumb ratios in f64[4].
3. **GPU Batching:** Route cell backgrounds into a background channel/pass, grid lines into an instanced hairline or fwidth shader pass, and cell text into either per-column merged draw channels (3 + D + 2N) or an instanced SDF/MSDF shader with per-instance clip_idx[1].
4. **Spatial & Selection Scaling:** Default to O(1) uniform math; promote to a 16-byte/node RLE Flat Segment Tree (mdds pattern) when users resize or hide individual rows[10]; and store selections in disjoint [2]i64 intervals with automatic promotion to a 1.19 MB bitset above 8,192 fragments[10].

Works cited

1. Building a High-Performance Spreadsheet Renderer - Quadratic AI, https://www.quadratichq.com/blog/building-a-high-performance-spreadsheet-renderer-why-we-chose-webgl-over-html
2. https://prompts.brightcoding.dev/blog/glide-data-grid-the-revolutionary-canvas-grid-for-react-apps
3. I moved virtualization math to Rust/WASM - here's what I learned, https://www.reddit.com/r/reactjs/comments/1q5kt5u/i_moved_virtualization_math_to_rustwasm_heres/
4. Virtual rendering for massive lists - Brian Simon, https://brianesimon.com/blog/virtual-rendering-massive-lists
5. Rendering Massive Tables at Lightning Speed: Virtualization with, https://dev.to/lalitkhu/rendering-massive-tables-at-lightning-speed-virtualization-with-virtual-scrolling-2dpp
6. https://www.1771technologies.com/blog/performance-benchmarks
7. imgui_tables.cpp - external/github.com/ocornut/imgui - Git at Google, https://skia.googlesource.com/external/github.com/ocornut/imgui/+/refs/heads/docking/imgui_tables.cpp
8. imgui/imgui_tables.cpp at master - GitHub, https://github.com/ocornut/imgui/blob/master/imgui_tables.cpp
9. Maximum element size is too restrictive for virtual scrolling [41441393], https://issues.chromium.org/41441393
10. unknown_url
11. imgui_internal.h - third_party/imgui - Git at Google, https://flutter.googlesource.com/third_party/imgui/+/25e5baf5f0da5aa0820b4fdc5b2633a7d25965aa/imgui_internal.h
12. Odin - Browse /dev-2026-05 at SourceForge.net,

https://sourceforge.net/projects/odin.mirror/files/dev-2026-05/
13. https://pkg.odin-lang.org/core/mem/virtual/
14. gpui - Rust - Docs.rs, https://docs.rs/xgpui/latest/gpui/
15. rune/PLAN.md at dev · anomalyco/rune - GitHub, https://github.com/anomalyco/rune/blob/dev/PLAN.md
16. Vision for Calc Development Keeping Calc Modular Kohei Yoshida, https://kohei.us/files/slides_libocon_2011.pdf
17. simd_csv - Rust - Docs.rs, https://docs.rs/simd-csv/latest/simd_csv/
18. package simd - pkg.odin-lang.org, https://pkg.odin-lang.org/core/simd/
19. package fmt - pkg.odin-lang.org, https://pkg.odin-lang.org/core/fmt/
20. https://blog.csdn.net/gitblog_00839/article/details/160576350
21. I embedded Node.js in a native program to render JSX without a, https://www.reddit.com/r/javascript/comments/1x5sico/i_embedded_nodejs_in_a_native_program_to_render/
22. Fun with SDL3 and Clay (3) - Dylan Cobb, https://dylancobb.dev/blog/clay-sdl-setup-3/
23. maxfridbe/csclay: A c# port of clay.h · GitHub - GitHub, https://github.com/maxfridbe/csclay
24. Karl Zylinski - Understanding The Odin Programming Language, https://www.scribd.com/document/1022515522/Karl-Zylinski-Understanding-the-Odin-Programming-Language-2024-Libgen-li
25. imgui_tables.cpp - external/github.com/ocornut/imgui - Git at Google, https://skia.googlesource.com/external/github.com/ocornut/imgui/+/3fef132f6285303af2784b2bd05b5b240b644379/imgui_tables.cpp
26. Bring_me_the_2D/external/imgui/imgui_internal.h - GitLab du LIRIS, https://gitlab.liris.cnrs.fr/rsoares/bringmethe2d/-/blob/master/Bring_me_the_2D/external/imgui/imgui_internal.h
27. Learning memory management - looking for review/critique - Explain, https://forum.odin-lang.org/t/learning-memory-management-looking-for-review-critique/437
28. package runtime - pkg.odin-lang.org, https://pkg.odin-lang.org/base/runtime/
29. Google Sheets Row Limit: Max Rows, Cell Caps & Fixes | Fastio, https://fast.io/resources/google-sheets-row-limit/
30. egui_extras: Improve the auto-sizing of `Table` (#4756, https://git.skyler.io/skyler/egui/commit/8ef0e85b85c210e0eb89a15f5a4dd2dfcc842f58?files=crates%2Fegui_demo_lib%2Fsrc
31. egui/crates/egui_demo_lib/src/demo/table_demo.rs at, https://git.skyler.io/skyler/egui/src/commit/be944f09153e3bf809c9a982004dc8ea8f153893/crates/egui_demo_lib/src/demo/table_demo.rs
32. Data Grid Virtual Scrolling in JavaScript Explained - Simple Table, https://www.simple-table.com/blog/data-grid-virtual-scrolling-javascript
33. ImDrawListSplitter::Merge - corrupted data in buffer (64k+ vertices, https://github.com/ocornut/imgui/issues/3129
34. imgui_draw.cpp - external/github.com/ocornut/imgui - Git at Google, https://skia.googlesource.com/external/github.com/ocornut/imgui/+/4691fa0ed5fa

b7ce25733e1b8d3d62ffc6511940/imgui_draw.cpp
35. Lesson 2 - Draw a circle - An infinite canvas tutorial, https://infinitecanvas.cc/guide/lesson-002
36. Drawing shapes with canvas - Web APIs | MDN, https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial/Drawing_shapes
37. Drawing Sharp Lines on the Canvas - KIRUPA, https://www.kirupa.com/canvas/drawing_sharp_lines_canvas.htm
38. HTML5 Canvas drawings like lines are looking blurry - TutorialsPoint, https://www.tutorialspoint.com/article/html5-canvas-drawings-like-lines-are-looking-blurry
39. HTML5 Canvas – Crisp lines every time - mobtowers - WordPress.com, https://mobtowers.wordpress.com/2013/04/15/html5-canvas-crisp-lines-every-time/
40. Dear ImGui - Browse /v1.93.0 at SourceForge.net, https://sourceforge.net/projects/dear-imgui.mirror/files/v1.93.0/
41. Lesson 5 - Grid | An infinite canvas tutorial - AntV, https://antv.vision/infinite-canvas-tutorial/guide/lesson-005
42. Lesson 5 - Grid | An infinite canvas tutorial, https://infinitecanvas.cc/guide/lesson-005
43. Build your Own Virtual Scroll - Part I - DEV Community, https://dev.to/adamklein/build-your-own-virtual-scroll-part-i-11ib
44. Virtual Scrolling for Billions of Rows — Techniques from HighTable, https://rednegra.net/blog/20260212-virtual-scroll/
45. How to Implement Virtual Scrolling Beyond the Browser's Limit, https://dev.to/kohii/how-to-implement-virtual-scrolling-beyond-the-browsers-limit-16ol
46. Frontend System Design: Virtualization & Handling Large Data Sets, https://dev.to/zeeshanali0704/frontend-system-design-virtualization-handling-large-data-sets-29nf
47. JavaScript Grid: DOM Virtualisation, https://www.ag-grid.com/javascript-data-grid/dom-virtualisation/
48. Documentation | Virtual Scroll - PDan Blog, https://pdanpdan.github.io/virtual-scroll/docs/
49. Tag: mdds - Roundtrip to Shanghai via Tokyo, https://kohei.us/tag/mdds/
50. Author: Kohei Yoshida - Roundtrip to Shanghai via Tokyo, https://kohei.us/author/admin/
51. Fenwick or Binary Index Tree - Dev Genius, https://blog.devgenius.io/fenwick-or-binary-index-tree-959c75a2000a
52. Architecture & Competitive Position — vlist v1 docs, https://vlist.io/docs/v1/resources/roadmap
53. Scroll: Virtual list engine for TypeScript - Vielzeug, https://vielzeug.dev/scroll/
54. libreoffice-impress-26.2.5.1-bp161.1.5 RPM for x86_64 - RPMfind.net, https://www.rpmfind.net/linux/RPM/opensuse/16.1/x86_64/libreoffice-impress-26.2.5.1-bp161.1.5.x86_64.html

55. What Is an R-Tree? A Complete Guide to Spatial Indexing - Lizardtech, https://www.lizardtech.com/post/what-is-an-r-tree-a-complete-guide-to-spatial-indexing
56. Spatial Index: R Trees - Towards Data Science, https://towardsdatascience.com/spatial-index-r-trees-5ac6ad36ca20/
57. Algorithms — list of Rust libraries/crates // Lib.rs, https://lib.rs/algorithms
58. imgui_demo.cpp - external/github.com/ocornut/imgui - Git at Google, https://skia.googlesource.com/external/github.com/ocornut/imgui/+/master/imgui_demo.cpp
59. src/imgui/imgui_widgets.cpp · master - GitLab, https://gitlab.vci.rwth-aachen.de:9000/ptrettner/imgui-lean/-/blob/master/src/imgui/imgui_widgets.cpp
60. One Pass to Parse Them All: Fused Parallel CSV Processing, https://www.vldb.org/pvldb/vol19/p3579-ellmann.pdf
61. jQuery Grid Scroll Modes Virtual Scrolling - Telerik.com, https://www.telerik.com/kendo-jquery-ui/documentation/controls/grid/scrolling/virtual-scrolling
62. Mistakes and cool things to do with arena allocators | Karl Zylinski, https://zylinski.se/posts/dynamic-arrays-and-arenas/